

claude-windows / 45ea3a91-654d-4972-9cd3-65f35db54caf

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\45ea3a91-654d-4972-9cd3-65f35db54caf\subagents\workflows\wf_2a64bc88-2a8\agent-aa8dcb214f58836cd.jsonl`  
- SHA-256: `1ae9cd742680939c56b5587c136cf1cce744344c4b8bb90163fba562de3c0842`  
- Source modified: `2026-06-10T06:13:47+00:00`  
- Imported at: `2026-07-05T16:48:27+00:00`  
- project: `wf_2a64bc88-2a8`  
- session_id: `45ea3a91-654d-4972-9cd3-65f35db54caf`
```

Transcript

1. user (2026-06-10T06:11:43.773Z)

CONTEXT ? two sibling repos on a Windows machine:

- INDEXER: C:/Users/avidu/Projects/badminton-highlight-indexer ? local AI badminton(->racket-sports) highlight indexer + rally detector. FastAPI + SQLite + ffmpeg + GPU CV via WSL; Gemini = paid cloud AI. docs/ tree is rich. Current branch docs/next-steps-multisport (master = main branch).

- COLLECTOR: C:/Users/avidu/Projects/sports-data-collector ? golden-label acquisition/curation/ingestion CLI (system-of-record candidate for training corpus).

Project state (June 2026): scaffolding complete; owned-model roadmap agreed (docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md) but NO platform/owned-model implementation started; next committed platform slice = async job model. Licensing guardrail: MIT/Apache-2.0/BSD only for code+data+weights; paid LLMs are inference-only, never training-data sources. ENVIRONMENT RULES: this dev environment has NO GPU/torch/cv2 ? do NOT attempt to run the video pipeline or import torch/cv2 modules. Test suites are offline/fully-mocked and SAFE to run. You are READ-ONLY: never edit/create/delete repo files; running tests and git read commands is allowed.

QUALITY BAR: cite file paths (and line numbers where possible) for every finding. Report what IS in the code/docs, not what you assume. Be skeptical and concrete. Your final structured output is consumed programmatically by an orchestrator ? return raw data, not prose for a human.

ROLE: adversarial verifier. An auditor ("docs-technical") claimed the finding below. Try to REFUTE it against the actual files. Default to isReal=false if the evidence does not hold up, the behavior is documented/deliberate (e.g. listed in docs/CODE_MAP.md gotchas or DEFERRED_HARDENING_PLAN.md as a known deferral ? unless the auditor adds genuinely NEW consequence), or the severity is materially overstated.

FINDING: {"title":"README Key Features + Project Structure block still describe the pre-#40 flat layout (6 of 12 entries wrong)","severity":"high","area":"README.md (repo root)","file":"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\README.md:19,326-345","evidence":"Line 19: 'Three Pluggable Rally Segmenters ('backend/indexer/')' ? actual: backend/pipeline/segmenters/ (and there are 5 registered segmenters: motion, gemini, yolo_hybrid, tracknet_hybrid, wasb_hybrid per backend/pipeline/segmenters/__init__.py, which line 219's Q7 acknowledges, contradicting line 19's 'Three'). Structure block lists `database.py` (actual backend/infrastructure/database.py), `validators/` (actual pipeline/validators/), `indexer/` (actual pipeline/segmenters/), `detectors/` (actual pipeline/detectors/), `stitcher/compiler.py` (actual pipeline/stitcher/compiler.py), `static/` (actual backend/api/static ? main.py:47 mounts StaticFiles(directory='backend/api/static')). config/, infrastructure/, reporting/, annotations/, tools/, pipeline/ are absent from the block.", "why_it_matters":"The repo-root README is the first navigation surface; half its map is wrong, and the leftover empty dirs make the wrong paths plausible. CODE_MAP is correct but a newcomer reads README first."}

Steps: open the cited file(s) and any counter-evidence files; verify quotes/line claims; judge severity honestly. Cheap greps allowed; do NOT run the pipeline. Return isReal, confidence, and a 1-3 sentence verdict (with corrected detail if partially right).

2. user (2026-06-10T06:11:48.953Z)

```

1      # Badminton Highlight & Rally Indexer
2
3      A local, AI-powered web application and CLI utility designed to process long-form
badminton recordings (e.g., from GoPro), run comprehensive pluggable validations (quality,
integrity, and relevance checks), index all rallies into a relational SQLite database, and
dynamically compile custom highlight reels using high-speed FFmpeg stitching.
4
5      The pipeline is built around three pluggable registries ? validators,
segmenters, and AI providers ? plus a sport profile registry so the same engine can
target other racket/court sports in the future. Badminton is the default profile.
6
7      > Where this is heading: the project is evolving from an AI-API-dependent prototype
into a self-improving tool that learns rally detection offline from a commercially-licensed,
annotated corpus. See [docs/ROADMAP.md](docs/ROADMAP.md) for the four-phase plan and
[docs/archive/decisions/DECISIONS.md](docs/archive/decisions/DECISIONS.md) (historical ADRs)
for the rationale.
8
9      ---
10
11     ## ? Key Features
12
13     * Pluggable Video Validation Framework: Easily extendable base-class architecture
that automatically runs checks on:
14         * Static Format & Resolution: Verifies container is MP4, resolution >= 720p, and
framerate >= 30 FPS.
15         * PTS Timeline Continuity: Scans keyframe timestamps for timeline gaps, jumps,
or editing splices to detect tampering or splices.
16         * Scene Cut Density Check: Measures sub-sampled HSV frame histogram correlation
to verify visual cuts. A continuous raw camera stream should have near 0 cuts.
17         * Blurriness Check: Calculates average frame Laplacian variance to reject out-
of-focus footage.
18         * Relevance (Is it Badminton?): Uses high-performance HSV color court matching
to identify court layouts.
19     * Three Pluggable Rally Segmenters (`backend/indexer/`):
20         * `yolo_hybrid` (default) ? two-stage pipeline. Stage 1 runs torchvision
person detection (Faster R-CNN, BSD-licensed) with ByteTrack tracking (supervision, MIT-
licensed) to cheaply filter the video down to a small set of high-motion candidate windows.
Stage 2 hands each candidate (with a small padding) to the AI provider for high-precision
semantic boundaries and shot counts. Massively cheaper than running the LLM on the full video.
(The registry key remains `yolo_hybrid` for back-compat; the AGPL-licensed YOLOv8 was replaced
with permissive components ? see
[docs/archives/MONETIZATION_AUDIT.md](docs/archives/MONETIZATION_AUDIT.md) and ADR-005.)*
21         * `gemini` ? sends the entire video (or a `chunk_duration` slice) to Google Gemini
directly. Highest precision per call, highest cost.
22         * `motion` ? zero-dependency OpenCV pixel-difference heuristic. No API key
required. Useful as a baseline or for offline/air-gapped use.
23     * Pluggable AI Provider Registry (`backend/providers/`): The Gemini provider ships
in-tree; new providers (e.g. an OpenAI or local-model backend) can be registered without
touching the segmenters.
24     * Pluggable Sport Profile Registry (`backend/sports/`): The per-sport LLM prompt
and relevance config live in a `SportProfile`. Add a new sport by subclassing `SportProfile` and
registering it.
25     * SQLite Indexing: Each processed video is keyed by MD5 hash. Play segments
(start/end/duration/server/receiver/metadata) and per-segment events are persisted in
`output/sports_indexer.db`. Re-running on the same file is a cache hit.
26     * Query System: Filter indexed play segments by `server`, `receiver`,
`duration_min`, and `event_type` (the schema supports rich event types like
smashes/serves/fouls; today the shipped segmenters mainly populate timing + estimated
`shot_count`).
27     * Zero-Dependency Local UI: Served statically directly by FastAPI (Vite + Node.js
NOT required!) featuring a glassmorphic dashboard, responsive filtering, and immediate clip
stitching.
28     * High-Speed FFmpeg Stitching with configurable `begin_padding`, `end_padding`,
and `min_shot_count` filtering.
29     * Detailed CLI Debugging Tools: `--debug-clip <t>` inspects frame metrics,

```

```

validator scores, and motion values around a timestamp. `--trim-start/--trim-end` extracts an
arbitrary segment via stream-copy ffmpeg for fast iteration.
30
31     ---
32
33     ## ?? Setup & Diagnostics
34
35     ### 1. External System Requirements
36     The indexer depends on **FFmpeg** and **FFprobe** to parse media containers, extract
keyframes, and stitch clips.
37
38     * **On Windows (PowerShell as Admin)**:
39     ```powershell
40     winget install Gyan.FFmpeg
41     ```
42     *Restart your terminal afterwards to register the path changes.*
43
44     ### 2. Auto-Installer & Diagnostics Checker
45     Run the diagnostics checker script in the root directory to verify your dependencies,
initialize default configs, and install PIP packages:
46     ```bash
47     python setup.py
48     ```
49     This script will:
50     * Validate `ffmpeg` and `ffprobe` are present in your PATH.
51     * Initialize a default `config.json` (if one does not exist) with thresholds for blur,
court HSV bounds, motion signals, YOLO velocity, chunking, and stitching.
52     * Detect whether an NVIDIA GPU is available (via `nvidia-smi`) and select the matching
PyTorch wheel index automatically (`cu124` if GPU present, `cpu` otherwise).
53     * Install everything from `requirements.txt`: `fastapi`, `uvicorn`, `opencv-python`,
`numpy`, `pydantic`, `jinja2`, `python-dotenv`, `google-genai`, `torch`, `torchvision`,
`supervision`.
54
55     > The default `yolo_hybrid` segmenter uses **torchvision**'s COCO-pretrained Faster
R-CNN, whose weights are downloaded and cached automatically by torchvision on first run ?
nothing is bundled in the repo. (Pick a lighter/heavier detector via `indexing.detector_model`:
`fasterrcnn_resnet50_fpn` (default), `fasterrcnn_mobilenet_v3_large_fpn`, or
`retinanet_resnet50_fpn`.)
56
57     ---
58
59     ## ? Running the Application
60

```

3. user (2026-06-10T06:11:50.155Z)

```

300     pytest                                # run everything
301     pytest tests/test_yolo_hybrid.py        # run a single file
302     pytest -k yolo --maxfail=1 -v           # filter by keyword, fail fast, verbose
303     pytest --cov=backend --cov-report=term-missing # coverage report (uses .coveragerc)
304     ```
305
306     > The optional `obsreport` package powers the per-video observability reports; its tests
307     > skip cleanly if it isn't installed. To exercise them: `pip install -e ../sports-
obsreport --no-build-isolation`.
308
309     The suite mocks external dependencies (the Gemini client, ffprobe/ffmpeg, OpenCV
`VideoCapture`, the YOLO model) so it runs offline and without a GPU. The tests cover:
310
311     * `tests/test_base.py` ? segmenter registry
312     * `tests/test_database.py` ? SQLite schema, video/segment insert/query
313     * `tests/test_geometry.py` ? bbox center / distance / velocity / IoU helpers
314     * `tests/test_video.py` ? ffprobe duration + ffmpeg chunk extraction helpers
315     * `tests/test_providers.py` + `tests/test_provider_gemini.py` ? AI provider registry
and the Gemini provider

```

```

316      *   `tests/test_sport_profiles.py` ? sport profile registry + the badminton prompt
317      *   `tests/test_gemini.py` ? `GeminiPlaySegmenter` (single-file and chunked paths)
318      *   `tests/test_yolo_hybrid.py` ? `HybridYoloSegmenter` happy path, missing-API-key
bailout, and timestamp-validation drops
319      *   `tests/test_tracknet_runner.py` + `tests/test_tracknet_hybrid.py` ? TrackNet WSL
runner (mocked subprocess) and the `tracknet_hybrid` segmenter
320      *   `tests/test_eval_metrics.py` + `tests/test_eval_annotations.py` +
`tests/test_eval_harness.py` ? rally evaluation harness (interval matching, CSV loading, DB
scoring, CLI)
321
322      ---
323
324      ## ? Project Structure (high level)
325
326      ```
327      backend/
328          main.py                # FastAPI app + CLI orchestrator
329          database.py            # SQLite schema and queries
330          validators/            # Pluggable video validators (format, blur, scene-cut,
relevance, ...)
331          indexer/              # Pluggable rally segmenters: motion, gemini, yolo_hybrid,
tracknet_hybrid
332          detectors/            # TrackNet WSL2 runner (shuttle-trajectory subprocess adapter)
333          providers/            # Pluggable AI providers (gemini today)
334          sports/               # Pluggable sport profiles (badminton today)
335          eval/                 # Rally evaluation harness: metrics, annotations, scoring
336          stitcher/compiler.py   # FFmpeg-based highlight compiler
337          utils/                # ffprobe/ffmpeg + bbox geometry helpers
338          static/               # Zero-build frontend served by FastAPI
339      docs/                     # Live docs (CODE_MAP, COMMERCIALIZATION, BACKLOG, ROADMAP,
guidelines, golden-set). Historical items in docs/archives/
340      tests/                   # pytest suite (mocked, runs offline)
341      run_process_and_stitch.py # Interactive end-to-end runner
342      run_eval.py              # Rally evaluation CLI (see "Annotation & Evaluation" above)
343      setup.py                 # Dependency & FFmpeg diagnostics + default config init
344      config.json              # Runtime configuration (see "Threshold Tuning" above)
345      ```
346

```

4. user (2026-06-10T06:11:58.169Z)

Name	Mode
----	----
__pycache__	d----
annotations	d----
api	d----
audio	d----
config	d----
detectors	d----
eval	d----
indexer	d----
infrastructure	d----
pipeline	d----
providers	d----
reporting	d----
sports	d----
stitcher	d----
tools	d----
utils	d----
validators	d----
main.py	-a---
results.py	-a---

5. user (2026-06-10T06:11:58.181Z)

```

205     ``bash
206     # Default: cheap candidate filtering with YOLO, then precise boundaries via Gemini
207     python -m backend.main --video-path "C:/path/to/video.mp4" --segmenter yolo_hybrid
208
209     # Full-video Gemini analysis (more expensive, highest semantic precision per call)
210     python -m backend.main --video-path "C:/path/to/video.mp4" --segmenter gemini
211
212     # Pure CV baseline ? no API key needed
213     python -m backend.main --video-path "C:/path/to/video.mp4" --segmenter motion
214     ``
215     *Or set `"default_segmenter": "yolo_hybrid"` (or `"gemini"`/`"motion"`) under
    `"indexing"` in `config.json`.*
216
217     The AI provider used by `yolo_hybrid` and `gemini` is controlled by `ai_provider` +
    `ai_model` at the top level of `config.json`. Today only `gemini` is registered; new providers
    can be added under `backend/providers/`.
218
219     **Trajectory-based segmenters (`wasb_hybrid`, `tracknet_hybrid`).** The registry also
    includes two shuttle-trajectory variants. Like `yolo_hybrid`, they cheaply filter the video down
    to candidate windows *locally* and then route each window to the AI provider (`gemini`) for
    precise boundaries ? so they need the same `GEMINI_API_KEY`. **If an observability report shows
    `segmenter_actual: wasb_hybrid` (or `tracknet_hybrid`), that is a valid, registered segmenter ?
    not an error.** Pick one with `--segmenter wasb_hybrid` or via `default_segmenter`.
    (`wasb_hybrid` additionally requires the WSL WASB setup ? see
    [docs/TRACKNET_WSL_SETUP.md](docs/TRACKNET_WSL_SETUP.md).)
220
221     ### Q8: How does the `yolo_hybrid` pipeline actually save money?
222     Sending a multi-gigabyte video to a hosted LLM is slow and expensive. `yolo_hybrid` runs
    in four phases:
223
224     1. **Chunking** ? splits the video into overlapping windows (`chunk_duration`,
    `chunk_overlap`) so we can process incrementally.
225     2. **Detection + tracking** ? torchvision person detection (Faster R-CNN) + ByteTrack
    runs on each chunk locally (GPU-accelerated when available), measuring per-track velocity.
    Frames whose normalised player velocity exceeds `yolo_velocity_threshold` are kept; the rest are
    dropped. Adjacent active frames are merged into candidate windows, with
    sub-`min_action_window_duration` windows discarded.
226     3. **AI handoff** ? only the surviving candidate windows (padded by `ai_handoff_padding`
    seconds on each side) are extracted and sent to the AI provider for precise rally boundaries and
    shot-count estimation. These calls run in parallel up to `ai_max_workers`.
227     4. **DB population** ? confirmed segments are written to `play_segments` along with
    `shot_count`, `shot_count_confidence`, and a `detector` tag like `yolo-hybrid-gemini-2.5-flash`.
228
229     In practice this typically cuts AI-provider runtime/cost by an order of magnitude versus
    uploading the full video to `gemini` directly, while preserving the LLM's strength at semantic
    boundary detection.
230
231     ### Q9: How can I debug/test the Gemini Segmenter on a small portion of a video without
    uploading a huge multi-GB file?
232     Ingesting and uploading multi-gigabyte video files to Google Cloud can be slow and
    expensive. We built a native **fast local trimming** developer debugging feature:
233     1. Open your `config.json` file.
234     2. Under the `"indexing"` configuration block, add a `"debug_duration"` key (in
    seconds):

```

6. user (2026-06-10T06:12:08.009Z)

indexer:

Name

__pycache__

validators:

__pycache__

detectors:

__pycache__
stitcher:
__pycache__
pipeline:
audio
detectors
segmenters
stitcher
validators

7. user (2026-06-10T06:12:08.153Z)

```
1:from backend.pipeline.segmenters.base import segmenter_registry, BasePlaySegmenter
2:from backend.pipeline.segmenters.motion import MotionPlaySegmenter
3:from backend.pipeline.segmenters.gemini import GeminiPlaySegmenter
4:from backend.pipeline.segmenters.yolo_hybrid import HybridYoloSegmenter
5:from backend.pipeline.segmenters.tracknet_hybrid import TrackNetHybridSegmenter
6:from backend.pipeline.segmenters.wasb_hybrid import WasbHybridSegmenter
10:    "BasePlaySegmenter",
11:    "MotionPlaySegmenter",
12:    "GeminiPlaySegmenter",
13:    "HybridYoloSegmenter",
14:    "TrackNetHybridSegmenter",
15:    "WasbHybridSegmenter",
```

8. user (2026-06-10T06:12:21.061Z)

False
True
False
True

9. user (2026-06-10T06:12:21.226Z)

```
23:from fastapi.staticfiles import StaticFiles
45:# Serves static frontend files directly (now under api/ per approved structure)
46:os.makedirs("backend/api/static", exist_ok=True)
47:app.mount("/static", StaticFiles(directory="backend/api/static"), name="static")
51:    index_path = "backend/api/static/index.html"
55:    return "<h3>Sports Highlight Indexer Server is Running. Please create index.html in
backend/api/static/</h3>"
```

10. assistant (2026-06-10T06:12:26.726Z)

All factual claims verified. Now checking whether this is already a documented/known deferral.

11. user (2026-06-10T06:12:28.776Z)

docs\DEFERRED_HARDENING_PLAN.md:39: shadows the build system; update
README/\`run\_all\_tests.py`\CI references.
docs\CODE_MAP.md:323:| \`test\_reporting\_spec\_faq.py`\ | \`reporting/spec.yaml\` <-> \`README.md\`
cross-ref integrity (every \`faq\_ref\` resolves; actionable rules cite a \`faq\_ref\`); ?
\`importorskip("obsreport")\` |
docs\README.md:25:See \`archives/README.md\` for the umbrella policy and
\`archives/past\_projects/README.md\` for classification of small tech debt / starter projects.
docs\archives\README.md:29:See \`past\_projects/README.md\` for the specific guidance on that
category.
docs\archives\checkpoints\2026-06-eval-baseline\README.md:30:- See \`../quality-
iterations/2026-06-initial/README.md\` for the phase this baseline seeds.
docs\archives\past_projects\low-regret-moves\LOW_REGRET_MOVES_PLAN.md:153:- Update all imports,
entry points, docs (\`CODE\_MAP.md\`, README, tests), and any hard-coded paths.
docs\archives\quality-iterations\README.md:7:See \`../README.md\` (under archives/) for the
overall archival policy.
docs\archives\quality-iterations\README.md:12: - A README.md with STATUS, trigger (new data,

specific goals), key changes, and links to relevant checkpoints under `../checkpoints/`. docs\archives\quality-iterations\2026-06-human-gt-tuning\README.md:139:- Seed: `../2026-06-initial/README.md`

12. user (2026-06-10T06:12:30.404Z)

329:| ? stale `.pyc` only | `tests/\_\_pycache\_\_/test\_gemini\_api.\*` ? no live source exists |

13. user (2026-06-10T06:12:39.583Z)

```
30     `requirements.txt`, with optional-dependency groups:
31     - `[project.optional-dependencies] dev` ? `pytest`, coverage tooling.
32     - `reporting` ? `obsreport` pinned to its git tag (`v0.1.3`, per BACKLOG).
33     - `gpu` ? torch/torchvision (note: the cu124 `--extra-index-url` is not
34       expressible in standard PEP 621 deps; document the install line or use a
35       `[tool.uv]/pip-conf` approach rather than faking it).
36     - Console entry points: `indexer = backend.main:main` (so `indexer --server` works),
37       optionally eval drivers.
38     - Rename the diagnostic `setup.py` (e.g. `scripts/doctor.py`) so it no longer
39       shadows the build system; update README/`run_all_tests.py`/CI references.
40     - Choose a build backend (hatchling recommended) and confirm the `backend/` import
41       root stays valid (keep current layout; no src/ move required).
42
43     ### Risks
44     - `setup.py` rename touches docs and any muscle-memory invocations ? grep first.
```

14. user (2026-06-10T06:12:40.682Z)

```
130
131     ### 4.4 Migration & Rollout (Config PR scope)
132     - Create the `backend/config/` package.
133     - Update `setup.py` to use the new loader/saver.
134     - Update `main.py` (CLI + API paths) to use typed config (minimal surface at first).
135     - Add `tests/test_config.py`.
136     - Update `CODE_MAP.md` and this plan.
137     - Verify no behavior change on existing runs (`--setup`, small video processing).
138     - Deliberately do not move other directories in this PR.
139
140     ### 4.5 Success Criteria
141     - Typed config is the only way new code reads settings.
142     - Existing `config.json` files (and `setup.py` output) continue to work unchanged.
143     - `backend/config/` package exists and sets the pattern for the larger structure.
144     - One focused, reviewable PR.
145
146     ---
147
148     ## 5. Next High-Priority Item: Directory Structure Refactor (Separate Atomic PR)
149
150     After the Config PR lands and is verified:
151     - Create a dedicated branch/PR whose only job is the layout change.
152     - Move files into `api/`, `pipeline/`, `infrastructure/` etc.
153     - Update all imports, entry points, docs (`CODE_MAP.md`, README, tests), and any hard-
154       coded paths.
155     - Goal: one clean atomic change with no one else (Claude) needing to resolve conflicts
156       at the same time.
157
158     This item is now tracked at the top of the sequence and in BACKLOG.
159
160     ---
161
162     ## 6. Later Low-Regret Items (after the above two)
163     - Error / Result Contract standardization (small, high signal). ?
164     - Logging standardization. ?
```

164 - Packaging modernization. ?
165 - Detector Runner Abstraction (WSL de-risk). ? (feat/finish-detector-runner-abstraction)
166 - Unify hybrid segmenter logic.
167 - Input Quality UX surfacing.
168 - Async job model slice.
169

15. user (2026-06-10T06:12:52.572Z)

d2bd952 docs: restructure to archives/ umbrella + past_projects/ classification (per feedback)
5b39068 docs+tooling: regenerate CODE_MAP for layered layout + run_all_tests.py + Grok low-regret notes
22d6410 chore(test): update reporting/validator test imports for layered package layout (post-#40 restructure)
0355417 Merge pull request #40 from avidullu/refactor/approved-directory-structure
1e96e07 test: fix mock patch targets after structure refactor and config test assertions
8615534 refactor: adopt approved layered directory structure
f4c1b2d docs: checkpoint approved structure + sequencing decisions (Config first, atomic refactor follow-up)
9029415 docs: checkpoint approved structure + sequencing decisions (Config first, atomic refactor follow-up)

16. user (2026-06-10T06:12:52.720Z)

```
28: -> db.add_video(video_id, filepath, status, [r.dict()])
@backend/infrastructure/database.py
30: -> seg = segmenter_registry.get(seg_name)(db, global_config) ?
@backend/pipeline/segmenters/base.py::segmenter_registry (400 + available list if missing)
50:WasbHybridSegmenter.process_video @backend/pipeline/segmenters/wasb_hybrid.py
81:- `@backend/main.py` ? Thin CLI + FastAPI bootstrap (orchestrator; modes via FLAGS). Static
UI mounts `/static` from `backend/api/static`. Logging is initialized with `basicConfig` (INFO,
timestamped format); backend modules use `logger = logging.getLogger(__name__)` (replaces stray
prints in hot paths like motion, stitcher, wasb_infer, config loader). User-facing CLI summaries
may still use `print` for direct output.
105:### 2.1 Segmenters (under pipeline/segmenters) ?
106:- `@backend/pipeline/segmenters/base.py`
108:[Omitted long matching line]
109:- `@backend/pipeline/segmenters/__init__.py` ? imports all 5
(`motion,gemini,yolo_hybrid,tracknet_hybrid,wasb_hybrid`) = the de-facto registration manifest;
re-exports them + `segmenter_registry` via `__all__`.
110:[Omitted long matching line]
111:- `@backend/pipeline/segmenters/tracknet_hybrid.py::TrackNetHybridSegmenter` (name
`'tracknet_hybrid'`) ? copy-paste twin of wasb_hybrid; imports `TrackNetRunner`/`TrackNetConfig`
from `pipeline/detectors/tracknet_runner`; `indexing.tracknet.velocity_threshold(0.02)`;
`detection_params` adds `queue_length`. ? any P3/P4 fix must be applied to BOTH (and arguably
yolo_hybrid).
```

[Showing results with pagination = limit: 10]

17. user (2026-06-10T06:13:16.922Z)

providers : __init__.py, base.py, gemini.py
sports : __init__.py, badminton.py, base.py
eval : __init__.py, annotations.py, batch.py, calibrate_local.py, calibrate_wasb.py,
calibration.py, classifier.py, distill_local.py, eval_partial_labels.py,
export_wasb_segments.py, gemini_refine.py, harness.py, metrics.py, rally_seq_proto.py,
regression.py, training.py
utils : __init__.py, geometry.py, hashing.py, video.py

18. user (2026-06-10T06:13:43.602Z)

Structured output provided successfully

19. assistant (2026-06-10T06:13:47.890Z)

Verification complete.